

Experiment Book

Student experiment booklet | Bash in the course GitHub Codespace

Name: _____ Lab ID: _____ Section: _____

You are helping prepare a campus event. Your files contain a small mess, missing information, and clues. Learn the tools, investigate your own data, then deliver a correct event report.

Get ready

1. Sign in to your own GitHub account. Open the link below and create a **Lab 1 Codespace** on **main**, or resume your existing Lab 1 Codespace. This lab uses a new repository, separate from the earlier practice sheet.

<https://codespaces.new/nalinabrol/terminal-labs?quickstart=1>

Wait for setup to finish. If prompted, choose **Trust Folder & Continue**. A maximized **Bash terminal** opens automatically. It works on files inside your remote Codespace.

2. Start in the **terminal-labs** repository folder. The practice kit is already included. Type:

```
bash terminal-lab-1/start.sh
```

3. Enter your assigned lab ID. Run the exact **cd** command printed by the setup, then type **pwd**. Setup creates a separate folder and preserves an existing attempt.

My lab root (copy the complete absolute path printed by pwd):

cat displays a text file. Run **cat identity.txt**. Copy your values below. You will use them later. If setup is missing or fails, ask the instructor before continuing.

EVENT: _____ TOKEN: _____ ROOM: _____

How to work: Predict first. Type and run. Inspect what changed. Write short observations in the spaces. Use the command reference on page 11. Do the file work in the terminal, without Explorer edits or generated solution scripts.

Working boundary: Change only files inside your lab root. Delete only the named disposable items. Do not use **sudo** or **recursive/force** deletion. Keep the original evidence files.

Where am I, really?

Start: your lab root - the absolute path you wrote on page 1. Begin each experiment here unless instructed otherwise.

A **directory** is a folder. A **path** is its address. **pwd** prints your current address. **ls** lists entries. **cd** changes your current folder. A successful command may print nothing.

```
pwd
ls
ls -la
```

In **ls -la**, **-l** requests details and **-a** includes names beginning with a dot. These names are normally hidden by plain **ls**. Ignore the permissions columns today.

1A. Which extra named file appears? Do not answer `.` or `..`.

1B | Predict a journey, then take it

Before running the commands, predict the final folder. `.` means here. `..` means the parent folder. A relative path is interpreted from your current folder.

```
cd maze/gate/desk
cd ..
cd .
pwd
```

Prediction (last folder name): _____ Observed: _____

From where you are now, return to the lab root using **one cd command with a relative path**. Verify with **pwd**. The first slash of an absolute path means filesystem root. This is not your lab root.

My return command:

1C | Does failure move you?

```
cd nowhere-here
pwd
```

What error did you get, and did your location change?

1D | Can you get home and back?

Run **cd ~**, then **pwd**. The `~` shortcut means your home directory. Now return to your lab root using its **absolute path from page 1**.

Try completion: type **ls ident**, press **Tab**, then Enter. Press **Up Arrow** to recall that command. Edit it to **ls -lah identity.txt** and run. **-h** makes file sizes easier to read with **-l**.

A copy or a move?

mkdir creates a directory. **touch** creates an empty file if the name is new. **cp SOURCE DESTINATION** copies a file. **mv SOURCE DESTINATION** moves or renames a file. Both can overwrite an existing destination. Use the new names given here.

2A | Build a working area

```
mkdir work
mkdir work/drafts work/submission
touch work/drafts/empty.txt
ls -l work/drafts
```

How many bytes does empty.txt contain? _____

2B | Make a copy, then rename it

Predict how many copies will exist after the first command. Then run all three commands and inspect both locations.

```
cp practice/original.txt work/drafts/copy.txt
mv work/drafts/copy.txt "work/drafts/announcement revised.txt"
ls practice work/drafts
```

Original still present? _____ copy.txt still present? _____

The quotes keep the space inside **one path**. Without quotes, Bash treats the words as separate arguments. The quotes are not part of the filename.

2C | Change its location

Now make a **second** copy of practice/original.txt named **delivery.txt** inside work/drafts. Move delivery.txt into work/submission, keeping its name. Inspect both folders.

Write your copy command and move command:

2D | Touch does not mean write

Run **cat "work/drafts/announcement revised.txt"**. Run **touch** on that same quoted path, then display it again. Predict before trying: will it become empty?

Prediction: _____ Observed: _____

Can an error protect a file?

rm removes a file. **rmdir** removes an empty directory. These terminal operations generally do not send files to a Trash or Recycle Bin. First verify the path. Remove only the disposable items below.

3A | Set up a controlled experiment

```
mkdir work/disposable
touch work/disposable/delete-me.txt
touch work/keep.txt
ls -la work/disposable
```

Predict what **rmdir work/disposable** will do. Run it once. Read the error and inspect the directory again.

Prediction:

Observed error and what remained:

3B | Remove exactly what was authorised

Remove **only work/disposable/delete-me.txt** using **rm**. Then remove the now-empty **work/disposable** directory using **rmdir**. Leave **work/keep.txt** in place.

My two commands:

Verify with **ls -la work**. A command printing nothing is not sufficient evidence that the correct target changed.

My evidence that **keep.txt** survived:

3C | Learn to ask for help

Run **ls --help**. Locate the description of **-a**. Then run **help cd**. The **cd** command is built into Bash, so it has a different help route. If manual pages are installed, **man ls** opens a manual. Press **q** to exit it. If **man** is unavailable, use **ls --help**.

In your own words, what does **-a** change?

Where did my first line go?

cat displays a whole text file. **head -n 3** displays its first three lines. **tail -n 3** displays its last three lines. Neither changes the file.

```
head -n 3 practice/reading.txt
tail -n 3 practice/reading.txt
```

First displayed line number: _____ Last displayed line number: _____

4A | Compare adding with replacing

echo prints text. **>** sends output to a file, replacing its previous contents. **>>** adds output at the end. Both create a missing file. Use only the scratch file below for this experiment.

```
echo "first" > work/scratch.txt
echo "second" >> work/scratch.txt
cat work/scratch.txt
```

Before the next command: predict which lines will remain afterward.

```
echo "replacement" > work/scratch.txt
cat work/scratch.txt
```

Observed contents and why they changed:

Do not redirect output back into the file you are reading. For example, **cat file > file** can empty it before **cat** reads it.

4B | Create a journal without losing a line

Create **work/journal.txt**. Its first line must be **your lab ID only**. Append a second line containing exactly **Stage: investigation**. Display the file to verify exactly two lines. Use your own ID, not the word ID.

My create command and append command:

4C | Recover when the prompt disappears

Run **cat** with no filename. Type **hello**, then Enter. It echoes the line and keeps waiting for keyboard input. Press **Ctrl+C** to stop it and return to the Bash prompt. Now **pwd** should work normally.

Why was cat waiting?

A filename is not its contents

* matches zero or more characters in a filename. Bash expands an unquoted filename pattern before running the command. Ordinary * matching skips leading-dot names and does not cross / into subdirectories.

5A | Predict which files match

```
ls -la practice/wildcards
ls practice/wildcards/*.txt
```

List the .txt filenames printed by ls (filenames only):

Inspect the names above. Explain why **.notice-secret.txt**, **notice.csv**, and **old/notice-c.txt** are not in the expansion.

My explanation:

5B | Ask find to explore below a directory

```
find practice/wildcards -type f -name "*.txt"
```

find searches from the starting directory. **-type f** selects files. **-name** matches their names. Quote **"*.txt"** here so find receives the pattern and applies it during its search. find can discover hidden and nested files. Output order can vary.

Which two additional .txt files did find discover?

5C | Locate your event brief

Your event code is on page 1 and in identity.txt. Under **records**, locate the file named **brief-YOUR_EVENT.txt**. Replace YOUR_EVENT with your actual code. Display the found file to discover your assigned room.

My find command:

Actual path found:

Room in the brief: _____ Matches identity.txt? _____

How many matches is a line?

grep -F "text" FILE displays lines containing literal text. Matching is case-sensitive by default.

-n adds line numbers.

-i ignores letter case.

wc -l FILE counts newline characters. Every record in the practice files ends with a newline, so this gives their line count.

6A | Test three different questions

```
cat practice/labels.txt
grep -Fn "ERROR" practice/labels.txt
grep -Fin "ERROR" practice/labels.txt
wc -l practice/labels.txt
```

Case-sensitive matching lines: _____ Case-insensitive matching lines: _____

Total lines in the file: _____ Does ERROR ERROR produce one line or two? _____

grep returns the **whole matching line**. Counting matching lines is not the same as counting every occurrence of a word.

6B | Find your registration trail

Display identity.txt if you need your TOKEN. In **practice/registrations.txt**, find only the lines containing your token. Save the original matching lines, without line-number prefixes, to **work/submission/my-records.txt**. Then display and count the saved lines.

My command that saves the matching records:

.....

.....

How many saved records? _____ First recorded status? _____

6C | Is empty output an error?

```
grep -F "NOT_PRESENT" practice/registrations.txt
```

An empty result can mean no line matched. It does not always mean the command is broken. A missing input file instead produces an error message.

How would you check that you searched the intended file?

.....

Does the order of commands matter?

A **pipe** `|` sends the left command's output into the right command as input. The right command can read those incoming lines without a filename. `>` saves output to a file. `|` sends output to another command.

7A | Predict, then count

```
cat practice/sample.log
grep -F "ERROR" practice/sample.log | wc -l
```

Predicted count: _____ Observed count: _____

This pipeline counts matching lines. It does not count all lines in the original log.

7B | Swap the order

Predict both outputs before running them. The labels A and B below are not commands.

A: Select errors first, then take the last two.

```
grep -F "ERROR" practice/sample.log | tail -n 2
```

B: Take the last two log lines first, then select errors.

```
tail -n 2 practice/sample.log | grep -F "ERROR"
```

A returned _____ lines. B returned _____ lines. Why are the results different?

7C | Produce two report files

Save all ERROR lines from `practice/sample.log` into **work/submission/sample-errors.txt**. Then save the last two matching lines into **work/submission/latest-two.txt**. Use a pipeline for the second file. Inspect both results. Preserve the original log.

My pipeline that creates `latest-two.txt`:

Finally, run **`wc -l work/submission/sample-errors.txt`**, then **`wc -l < work/submission/sample-errors.txt`**. `<` supplies a file as standard input. Compare the output labels. The counts should agree.

Which version prints only the count, without the filename?

Prepare the event for opening

The coordinator has left a folder called **rescue**. Your job is to prepare the correct announcement and incident report. Use the tools you have practised. A clear multi-step solution is as valid as a short one.

Your mission

- 1. Discover.** Inspect **rescue**, including hidden entries. Find and read the dispatch instructions. They identify your event, the approved announcement filename, and the one disposable file you may remove.
- 2. Locate and preserve.** Find the approved announcement under **rescue/inbox**. There are decoys. Create **work/submission/rescue**. Copy the correct announcement there as **announcement final.txt**. Keep the original unchanged.
- 3. Filter.** Use **rescue/system.log**. Save **only lines that contain both your EVENT field and ERROR** into **work/submission/rescue/errors.txt**. Preserve the original lines and their order. Do not include other events or INFO lines.
- 4. Count and inspect.** Save the number of those error records, with no filename label, into **error-count.txt**. Save the last three matching records into **latest-three.txt**. Put both files in **work/submission/rescue**.
- 5. Clean up precisely.** Remove only the disposable file named by dispatch. Preserve the other files in **rescue/discard**. Inspect your final folder and reports.

Required final folder

```
work/submission/rescue/
  announcement final.txt
  errors.txt
  error-count.txt
  latest-three.txt
```

Hint: Use **two successive grep filters** when both conditions must be true. All files in this log end with a newline, and its order is chronological. Keep the inputs and output files separate.

Approved announcement source path:

My event error count: _____ Last error message: _____

Write the command(s) that selected both conditions:

EXIT CHECK

Can you explain your evidence?

1 | Run the feedback checker

From your lab root, run this exact command. It reads your files and reports what needs attention. It does not fix your work or submit it.

```
python3 ../../terminal-lab-1/check.py
```

If earlier pages are unfinished, some checks will say CHECK. Use the messages to locate the problem. The checker checks file contents and locations. Your instructor checks predictions and explanations.

File checks passed: _____ / 16 One issue I fixed or still need help with:

2 | Show a command and explain it

Choose a command or pipeline you actually used. Show it in the terminal and explain what each part does. Your instructor may ask you to change a token or starting location.

The command I chose:

What it reads, what it does, and where the result goes:

3 | Answer from your experiment

A classmate runs `tail -n 3 rescue/system.log | grep -F "ERROR"` and calls the result "the last three errors". Is that guaranteed? Explain using what you observed on page 8.

My explanation:

Before leaving

☐ I have filled in my name and lab ID. ☐ I have preserved my files.

☐ I can show my reports and explain a command. ☐ I have handed in this booklet as instructed.

Keep your Codespace and lab-work folder. Do not delete the environment after class. Show your terminal/files to the instructor. The checker does not upload anything.

Optional stretch | Only after the required work

Enter `maze/gate/desk`. Without changing directory again, display the first line of `identity.txt` using a relative path. Then do it with an absolute path. Explain why one address depends on where you started. Return to your lab root afterward.

The tools you used today

Examples below use generic names. Replace them with paths from your exercise. Do not type a \$ prompt, angle-bracket placeholders, or the explanatory text.

Command or notation	What it does
pwd ls ls -lah	pwd prints your location. ls lists entries. ls -lah adds details, hidden entries and readable sizes.
cd folder cd .. cd ~	cd folder enters a folder. cd .. goes to its parent. cd ~ goes home.
... / ~	. means here. .. means parent. / means filesystem root. ~ means home. A leading / makes a path absolute.
mkdir notes touch notes/new.txt	mkdir creates a folder. touch creates an empty file if absent. Otherwise, it updates timestamps.
cp source.txt copy.txt mv copy.txt "renamed copy.txt"	cp copies a file. mv moves or renames it. Quote paths containing spaces.
rm disposable.txt rmdir empty-folder	rm removes the named file. rmdir removes an empty directory. Check the target first.
cat notes.txt head -n 3 notes.txt tail -n 3 notes.txt	cat displays the whole text file. head -n 3 shows its first three lines. tail -n 3 shows its last three lines.
echo "ready" > status.txt echo "done" >> status.txt	The first command writes ready, replacing any existing contents. The second appends done on the next line.
*.txt	Filename pattern. Normally excludes leading-dot files. Does not search subfolders.
find records -type f find records -type f -name "*.txt"	The first command finds files below records. The second limits results to names matching the quoted pattern.
grep -F "READY" notes.txt grep -Fin "READY" notes.txt	Find lines containing literal text. -i ignores case. -n adds line numbers.
wc -l notes.txt wc -l < notes.txt	Both count newlines. The first includes a filename label. The second reads the file as input and prints only the count.
grep -F "READY" notes.txt wc -l	Pipe selected lines into the counter. The character here really is a pipe.
> >> <	> writes output and replaces contents. >> appends output to a file. < reads input from a file. passes output to the next command.
Tab / Up Arrow / Ctrl+C	Tab completes names. Up Arrow recalls a command. Ctrl+C interrupts a running command.
ls --help help cd man ls	ls --help shows help for ls. help cd shows Bash help for cd. man ls opens a manual if installed. Press q to quit.